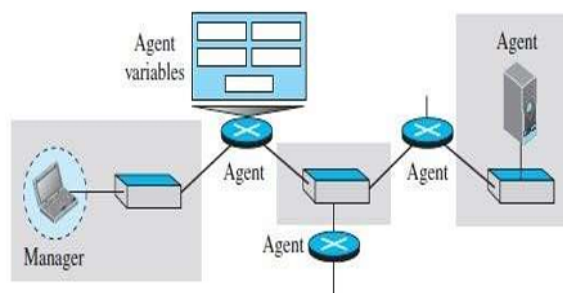


UNIT V APPLICATION LAYER

- ☐ Application layer-WWW and HTTP
- ☐ FTP
- ☐ Email
- ☐ Telnet
- ☐ SSH
- ☐ DNS
- ☐ SNMP



1 World Wide Web (WWW)

Origin & Idea

- Proposed by **Tim Berners-Lee (1989)** at CERN.
- Purpose: allow researchers across Europe to share work.
- Commercial web started in **early 1990s**.

Key Features

- **Distributed** → Any web server can add pages, helps growth.
- **Linked** → Pages can connect to other pages using **hypertext / hypermedia** (text, images, audio, video).

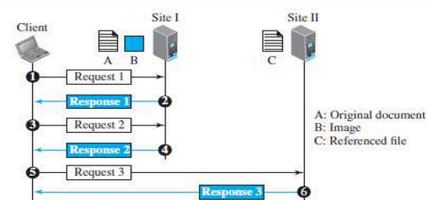
Uses

- Information sharing.
- Online shopping & gaming.
- Radio, TV, entertainment.

WWW Architecture

- **Client-Server model:**
 - Client = Browser (user side).
 - Server = Stores and sends web pages.
- **Sites** → Hold many web pages.
- **Page types:**
 - Simple (no links).
 - Composite (has links).
- Each page = **file with name & address.**

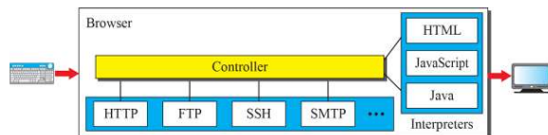
Figure 26.1 Example 26.1



1. Web Client (Browser)

- Examples: Chrome, Firefox, Internet Explorer.
- **Parts:**

- **Controller** → Takes user input, manages document.
- **Client Protocols** → HTTP, FTP.
- **Interpreters** → HTML, Java, JavaScript (to display).



2. Web Server

- Stores and serves web pages.
- Uses **cache** for speed.
- Supports **multithreading** (multiple requests at once).
- Examples: Apache, Microsoft IIS.

3. URL (Uniform Resource Locator)

- Every web page has a unique **address**.
- 4 parts:
 1. **Protocol** (HTTP, FTP).
 2. **Host** (domain name or IP).
 3. **Port** (default for HTTP = 80).
 4. **Path** (location of file in server).

4. Web Documents

1. Static Documents

- Fixed content, pre-stored.
- Languages: HTML, XML, XHTML, XSL.

2. Dynamic Documents

- Created fresh by server for each request.
- Example: showing time & date.
- Languages: CGI, JSP, ASP, SQL.

3. Active Documents

- Runs at **client side** (browser).
- Example: animations, interactive programs.
- Languages: Java Applets, JavaScript.

1.2 HyperText Transfer Protocol (HTTP)

Basics

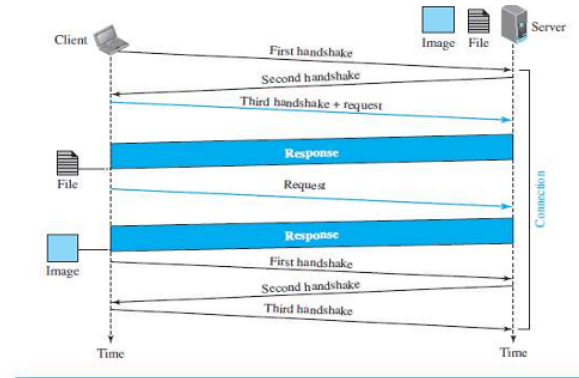
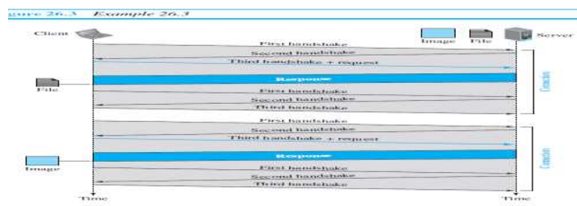
- HTTP = protocol to **get web pages** (client ↔ server).
- **Client** sends request, **server** sends response.
- **Port used**: 80 (default).
- Uses **TCP** → connection-oriented, reliable.
- Steps:

- Connection established → data transfer → connection closed.

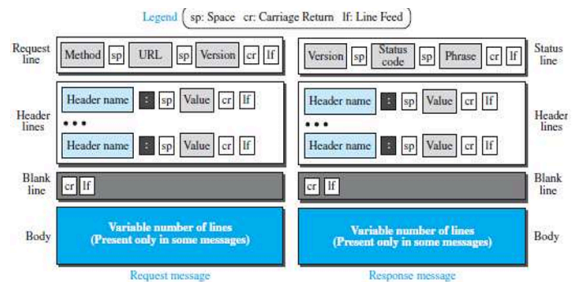
Connection Types

1. Nonpersistent (HTTP 1.0)

- Each request/response = new TCP connection.
- Overhead = too many connections, more memory, more handshakes.
- If a page has N pictures → needs N+1 connections.



HTTP Message Format



Request Message

- 3 parts in first line:
 1. **Method** (GET, POST, PUT, DELETE, etc.)
 2. **URL** (address of page).
 3. **Version** (e.g., HTTP/1.1).
- Can include **headers** (extra info from client).

2. Persistent (HTTP 1.1)

- Connection stays open for multiple requests.
- Saves time and resources.
- Server may close if client asks or timeout happens.
- Better for performance.

Table 1 Methods	
Method	Action
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

Table 2 Request header names	
Header	Description
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server
If-Modified-Since	If the file is modified since a specific date

Response Message

- 3 parts in first line:
 1. **Version** (HTTP/1.1).
 2. **Status Code** (3 digits).
 - 1xx → Info.
 - 2xx → Success.
 - 3xx → Redirect.
 - 4xx → Client error.
 - 5xx → Server error.
 3. **Status Phrase** (text form, e.g., "OK").
- Headers → extra info from server.
- May include **body** (actual data).

Table 3 Response header names	
Header	Description
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

Caching

- Cache = stores copies of web pages.
- **Benefits:**
 - Faster access for client.

- Less load on server.

- Cache rules (directives): defines if page can be cached, how long, etc.

Proxy Servers (Web Caching)

- Proxy server = middle computer that stores responses.
- Steps:
 - Client request goes to proxy.
 - Proxy checks cache.
 - If found → returns cached page.
 - If not → forwards request to main server.
- Advantages:
 - Reduces server load.
 - Less network traffic.
 - Faster response (low latency).

2 File Transfer Protocol (FTP)

♦ Basics

- FTP = standard TCP/IP protocol to **copy files between hosts.**
- Better than HTTP for **large files** or **different formats.**
- Solves problems:
 - Different file names, data formats, directory structures.

♦ Client & Server

- Client parts:
 1. User Interface

2. Client Control Process
3. Client Data Transfer Process

- **Server parts:**

1. Server Control Process
2. Server Data Transfer Process

◆ Two Connections

1. Control Connection

- Port **21**.
- Stays open during session.
- Commands (client → server) & responses (server → client).
- Commands = uppercase ASCII (e.g., USER, PASS, RETR, STOR).
- Responses = 3-digit code + text.

2. Data Connection

- Port **20**.
- Opens/closes for each file transfer.
- Used for actual file content.

◆ File Transfer Actions

- **Retrieve (download):** server → client.
- **Store (upload):** client → server.
- **Directory listing:** server → client.

◆ Communication Attributes

1. File Type

- ASCII file
- EBCDIC file

- Image file (binary)

2. Data Structure

- File structure → continuous bytes (default).
- Record structure → divided into records (for text).
- Page structure → divided into pages (with header & number).

3. Transmission Mode

- **Stream mode** (default): continuous bytes.
- **Block mode:** data in blocks (with 3-byte header).
- **Compressed mode:** reduced size for efficiency.

3 TELNET

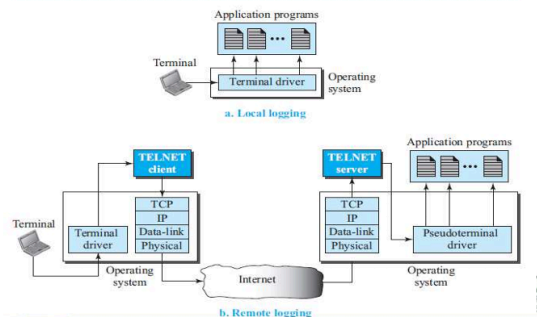
◆ Basics

- TELNET = **TErminaL NETwork** (remote login protocol).
- Lets user log in and use programs on a **remote computer**.
- **Weakness:** Sends data (including password) in **plain text** → insecure → mostly replaced by **SSH**.
- Still used by **network admins** for testing and debugging.

♦ Local vs Remote Logging

Local Logging

- User logs into own computer.
- Keystrokes → terminal driver → OS → runs program.

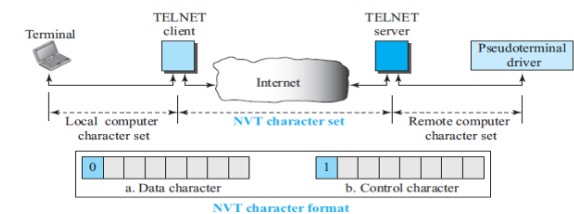


Remote Logging

- User logs into another computer.
- Steps:
 1. Keystrokes go to **TELNET client**.
 2. Client converts them into **NVT (Network Virtual Terminal)** characters.
 3. Sent via **TCP/IP** to remote machine.
 4. TELNET server converts NVT → format for remote OS.
 5. **Pseudo terminal driver** used → makes OS think input came from local terminal.
 6. OS runs requested program.

♦ Network Virtual Terminal (NVT)

- Solves problem of **different OS using different control characters**.
 - Example: DOS = Ctrl+Z, UNIX = Ctrl+D.
- NVT defines a **universal character set**.
- Two sets of characters:
 - **Data characters** → NVT ASCII (7-bit ASCII + 1 zero bit).
 - **Control characters** → 8-bit, highest bit = 1.



♦ Security Issue

- TELNET = **not encrypted** → anyone can sniff passwords & commands.
- Replaced by **SSH (Secure Shell)** for secure remote login.

4 Secure Shell (SSH)

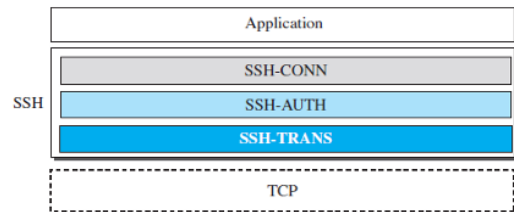
♦ Introduction

- SSH = **Secure Shell** → A secure application protocol.

- Replaced **TELNET** (because TELNET sends plaintext).
- Uses **encryption** to secure data.
- Two versions:
 - **SSH-1** → old, insecure, deprecated.
 - **SSH-2** → current, secure, widely used.

- Compression → makes transfer faster + harder to attack.

Components of SSH



◆ Components of SSH

1. SSH Transport-Layer Protocol (SSH-TRANS)

- Works **on top of TCP** (since TCP is not secure).
- Establishes a **secure channel**.
- Steps: TCP connection → exchange security parameters → create encrypted tunnel.
- Services it provides:
 - Privacy (confidentiality) → data is encrypted.
 - Integrity → ensures data is not modified by attackers.
 - Server authentication → ensures the server is genuine.

2. SSH Authentication Protocol (SSH-AUTH)

- Authenticates **client to server** (after server is verified).
- Similar to **SSL authentication**.
- Process:
 - Client sends → username, server name, authentication method, data.
 - Server replies → Success or Failure (retry if needed).

3. SSH Connection Protocol (SSH-CONN)

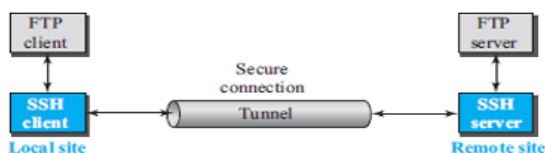
- Works after both sides are authenticated.
- Provides **multiplexing** → multiple logical channels over one secure connection.

- Each channel can be used for:

- Remote logging
- File transfer (SFTP, SCP)
- Other secure communication

◆ Applications of SSH

- **Remote Logging** → Programs like PuTTY, Tectia use SSH.
- **File Transfer** →
 - **SFTP (Secure File Transfer Protocol)** – uses SSH channels.
 - **SCP (Secure Copy)** – works like UNIX `cp` but secure.
- **Port Forwarding (SSH Tunneling)** →
 - Creates a secure tunnel for other protocols (e.g., FTP).
 - Protects insecure applications by routing traffic through SSH.

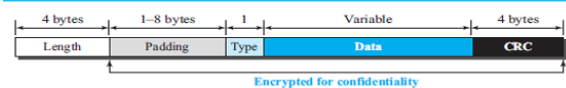


◆ SSH Packet Format

- **Length field** → size of the packet (excluding padding).

- **Padding (1–8 bytes)** → adds randomness, prevents attacks.
- **CRC (Cyclic Redundancy Check)** → error detection.
- **Type field** → defines the type of SSH message.
- **Data field** → actual message content.

Figure 26.27 SSH packet format



5 Electronic Mail (E-Mail)

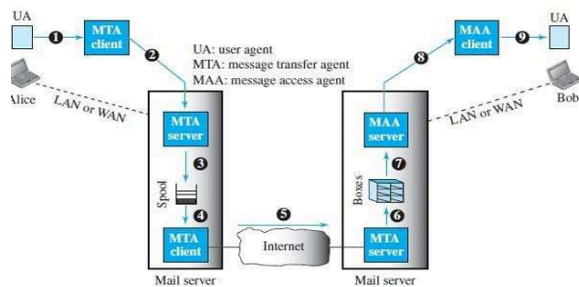
◆ What is Email?

- Electronic Mail = exchange of messages over the Internet.
- **Client–Server model** → users use clients, servers handle delivery/storage.
- Needs **mailbox (for storing received messages)** and **queue/spool (for outgoing mail)**.

◆ Email Architecture (Steps in sending mail)

1. User (Alice) writes message using **UA (User Agent)**.
2. UA sends it to **MTA Client**.
3. Stored in **queue (spool)** on mail server.

4. Sent by MTA to remote server's MTA.
5. Receiver's mail server stores in mailbox.
6. Receiver (Bob) uses **MAA Client** to retrieve message.
7. Bob reads using UA.



Key Agents:

- **UA (User Agent):** Compose, send, read (e.g., Outlook, Eudora).
- **MTA (Message Transfer Agent):** Pushes mail between servers (uses SMTP).
- **MAA (Message Access Agent):** Fetches mail for user (uses POP3 / IMAP).

♦ User Agent (UA)

- Software for sending/receiving emails.
- Types:
 - **Command-driven** (old: mail, pine, elm).

- **GUI-based** (modern: Outlook, Eudora).

- Functions: compose, read, reply, forward, manage mailbox.

♦ Sending Mail

- Structure:
 - **Envelope** (sender, receiver, delivery info).
 - **Message** → Header (sender, receiver, subject) + Body (actual content).

♦ Receiving Mail

- UA checks mailbox (manually or by timer).
- Shows list → sender, subject, time.
- User opens message to read contents.

♦ Email Address Format

- **local-part @ domain**
 - Local part → user's mailbox.
 - Domain → mail server host (e.g., Gmail, Yahoo).

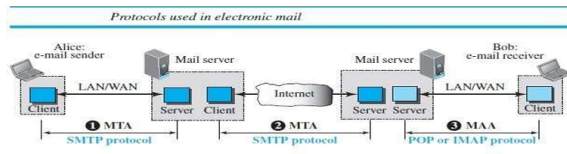
♦ Mailing List

- One alias name = multiple recipients.
- Example: class@college.edu → delivers to all class members.



5.1 SMTP (Simple Mail Transfer Protocol)

- Protocol for MTA communication.
- Used twice:
 1. User → Sender's mail server.
 2. Sender's mail server → Receiver's mail server.
- **Push protocol** (client pushes mail to server).
- Uses **Port 25 (TCP)**.



◆ Phases

1. Connection Establishment

- Server: 220 (ready) / 421 (not available).
- Client: HELO + domain.
- Server: 250 (OK).

2. Mail Transfer (8 steps)

- MAIL FROM (sender) → 250 OK.
- RCPT TO (receiver) → 250 OK.

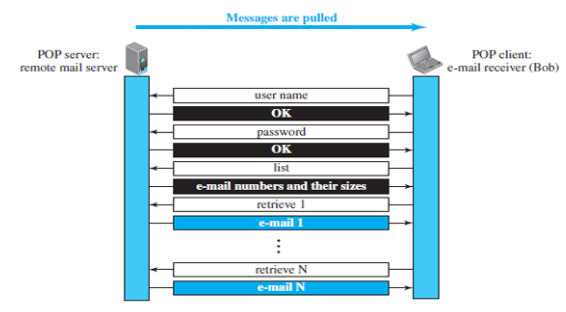
- DATA → 354 (start input).
- Message body + "." → 250 OK.

3. Connection Termination

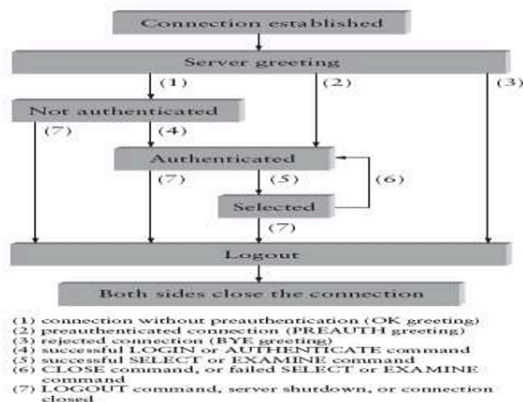
- QUIT → 221 (closing).

5.2 POP3 (Post Office Protocol v3)

- Retrieves mail from server → **downloads to client**.
- Uses **TCP Port 110**.
- Modes:
 - **Delete mode** → removes mail after download.
 - **Keep mode** → keeps copy on server.
- Commands:
 - USER, PASS → login.
 - STAT → no. of mails + size.
 - LIST → list of messages.
 - RETR → get message.
 - DELE → delete message.
 - QUIT → logout.



5.3 IMAP (Internet Message Access Protocol)



IMAP state transition diagram.

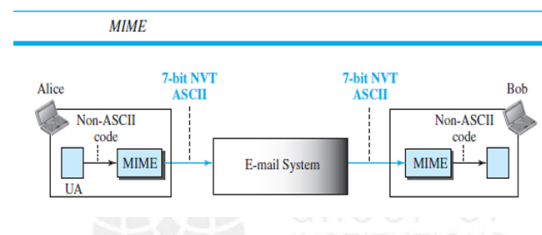
- More powerful than POP3.
- User can:
 - Check headers before download.
 - Search mail without full download.
 - Partially download messages (good for slow internet).
 - Create, delete, rename folders on server.
- Uses TCP.

IMAP > POP3 because:

- ✓ Keeps mail on server.
- ✓ Supports multiple devices.
- ✓ Folder management.

5.4 MIME (Multipurpose Internet Mail Extensions)

- Extends email → allows **non-ASCII data** (images, audio, video, files).
- Converts data → **ASCII (7-bit)** for safe transport.



- Headers added:
 - MIME-Version → version used.
 - Content-Type → type/subtype (text/plain, image/jpeg, audio/mp3, etc.).
 - Content-Transfer-Encoding → method (Base64, Quoted-Printable, etc.).

MIME header

E-mail header	
MIME headers	MIME-Version: 1.1
	Content-Type: type/subtype
	Content-Transfer-Encoding: encoding type
	Content-ID: message ID
	Content-Description: textual explanation of nontextual contents
E-mail body	

Base64 Encoding

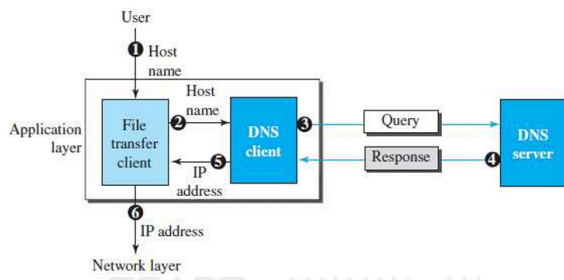
- Used in MIME to send binary files.

- Process:
 - 3 bytes (24 bits) → split into 4 × 6-bit chunks.
 - Each chunk → mapped to 1 ASCII char (64 total).
 - Valid chars = A-Z, a-z, 0-9, +, /.

6 Domain Name System (DNS)

♦ What is DNS?

- DNS = **Domain Name System Protocol**.
- Used to map names (like www.google.com) → IP addresses (like 142.250.183.206).
- Works like the **Internet's phonebook**.
- Needed because humans remember names, computers need IP addresses.



♦ Steps: How DNS Maps a Name to an IP Address

1. User gives hostname (e.g., "google.com") to an application (like file transfer client).
2. Client passes hostname → DNS client (resolver).
3. DNS client sends query → DNS server (whose IP it already knows).
4. DNS server finds & responds with IP address of requested host.
5. DNS client gives IP address → application (file transfer client).
6. Application uses IP address to connect to the server.

♦ Name Space

- Names must be unique (since addresses are unique).
- Two types:

1. Flat Namespace

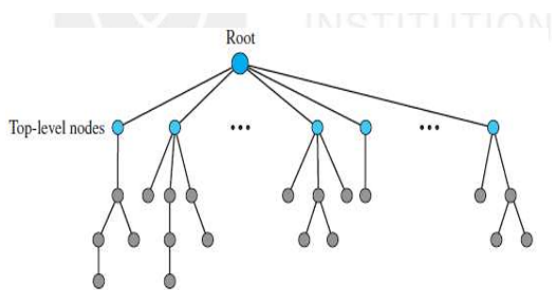
- Just a sequence of characters, no structure.
- Works only for small systems.
- Problem: duplication risk in large systems like Internet.

2. Hierarchical Name space

- Names broken into levels (organization → department → machine).
- Central authority controls main parts.
- More scalable → used in DNS.

♦ Domain Name Space

- Organized as **inverted tree structure** (root at top).
- Max **128 levels** (root = level 0).

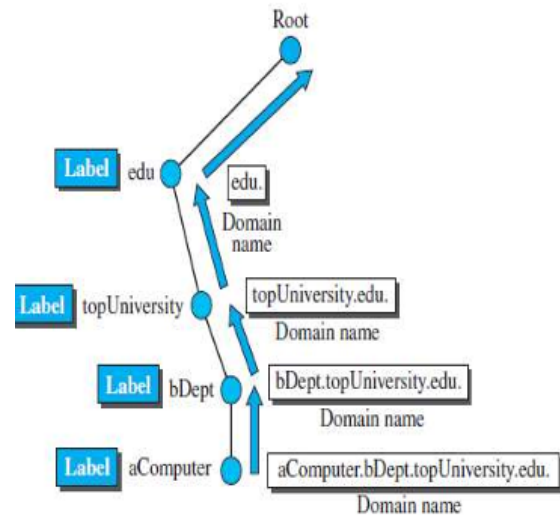


Label

- Each node = label (≤ 63 characters).
- Root label = empty string (null).

Domain Name

- Full name = sequence of labels separated by dots.
- Always read **from node** → **root**.
- Ends with a dot **.** (null label).



Types:

- **FQDN (Fully Qualified Domain Name):** Ends with **.** → e.g., www.example.com.
- **PQDN (Partially Qualified Domain Name):** Doesn't end with **.** → e.g., www.example.com

1. Domain

Domain = subtree of the domain name space. The name of the domain = top node of subtree.

2. Distribution of Name Space

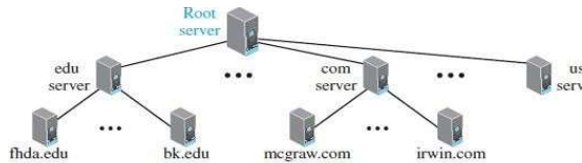
Zone info is stored in primary & secondary servers. Zone = portion of namespace controlled by server."

3. Hierarchy of Name Servers

Zone info is divided into domains → subdomains.

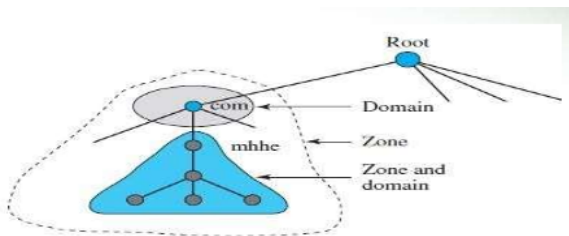
Servers are authoritative for large or small domains.

We have a hierarchy of servers like we have a hierarchy of names.



Domains & Zones

- **Domain** = subtree of the domain name space.
- **Zone** = portion of namespace controlled by a DNS server.



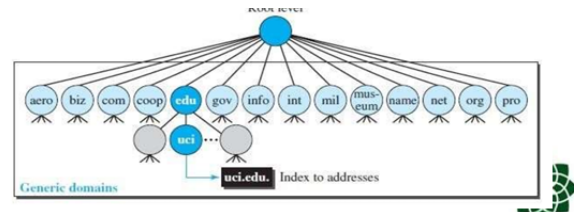
- Zone info is stored in **primary & secondary servers**.

Servers

- **Root Server** – knows about top-level domains (.com, .org, .in). Distributed worldwide.
- **Primary Server** – has original zone data (creates & updates zone files).
- **Secondary Server** – copies zone data from primary. Doesn't create or update.

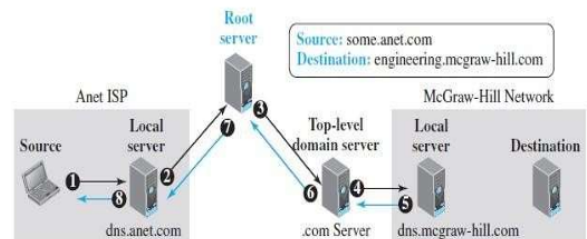
DNS in Internet

1. **Generic Domains** → based on type (.com, .org, .edu).
2. **Country Domains** → two letters (.in, .us, .uk).
3. **Inverse Domain** → maps IP → Name (reverse lookup).



Name Resolution

- Converting **name** ↔ **address**.
- DNS works in **client-server mode**.
- Resolver (client) contacts DNS servers.



Two Methods:

1. Recursive Resolution

- Each server contacted **keeps asking on behalf of client** until answer found.

- Final answer returned back step-by-step.
- Example: Client → ISP DNS → Root → TLD (.com) → Company DNS → returns IP.

2. Iterative Resolution

- Each server contacted replies with **next server's IP**.
- Client itself keeps asking next servers until it gets the answer.

7 SNMP (Simple Network Management Protocol)

- What it is:
 - Application layer protocol (works over TCP/IP).
 - Used for **monitoring & managing network devices** (routers, servers, etc.).
 - Works in a **Manager ↔ Agent** model.
- Manager:
 - A host running the SNMP client program (GUI).
 - Sends requests, receives responses.
- Agent:

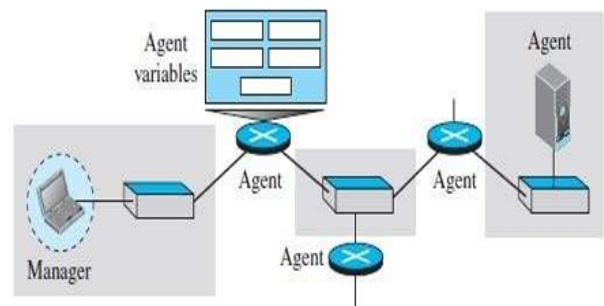
- A router or host running SNMP server program.
- Stores performance info in a **database**.
- Responds to manager queries.

- Ports:

- UDP 161 → Agent.
- UDP 162 → Manager.

- Operations:

- **GET** → retrieve value of variable.
- **SET** → change value of variable.
- **GET-NEXT** → move through tables.
- **RESPONSE** → reply from agent to manager.
- **TRAP** → event notification from agent to manager (e.g., reboot, error).



⚙️ SNMP Components

- **SNMP itself**: Defines packet format & message exchange.
- **SMI (Structure of Management Information)**: Defines rules for

naming, types, encoding.

- **MIB (Management Information Base):** Database of variables/objects for each device.



Role of SNMP

- Defines **packet formats** for manager ↔ agent.
- Reads & modifies variables (called **objects**).
- Creates **statistics & reports** (with other software help).

Role of SMI

- Provides **rules**, not actual data.
- Attributes handled:

1. **Name** → Each object has a unique **Object Identifier (OID)**.

- Hierarchical tree (root → branches).

- Example:
1.3.6.1.2.1
(numeric) or textual names.

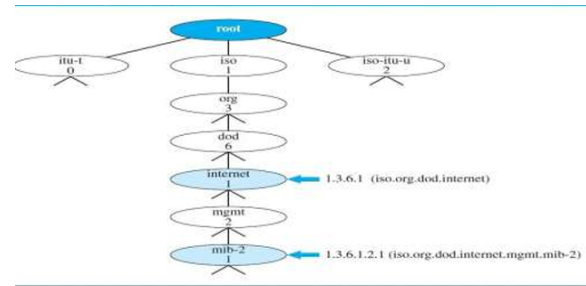
2. **Data Type** → Uses ASN.1 types.

- **Simple** (integer, string, etc.).

- **Structured** (sequence, sequence of).

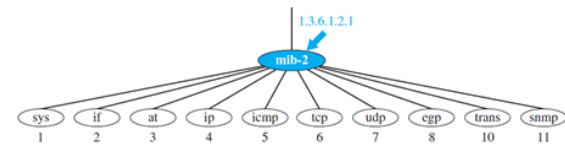
3. **Encoding** → Uses **BER (Basic Encoding Rules)**.

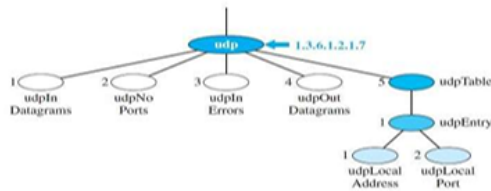
- Encodes data as **TLV (Tag, Length, Value)** format.



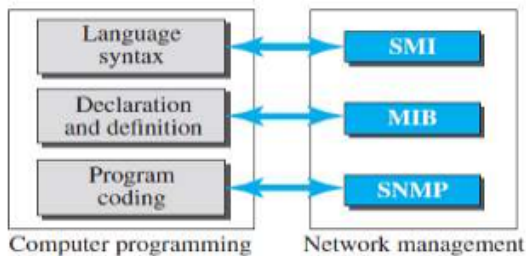
Role of MIB

- Actual collection of objects for each managed entity.
- Current version → **MIB-II**.
- Objects are grouped (like database tables). Examples:





- **sys** → System info (name, location, uptime).
- **if** → Interfaces info (number, IP, MAC).
- **at** → ARP table info.
- **ip** → Routing table, IP addresses.
- **icmp** → ICMP packets/errors.
- **tcp** → TCP connections, ports, timeouts.
- **udp** → UDP ports, packets sent/received.
- **egp** → Exterior Gateway Protocol stats.
- **trans** → Transmission methods (future use).
- **snmp** → Info about SNMP itself.



Accessing MIB Variables

- Manager sends identifier for variable → Agent looks it up → Sends back encoded value.

- Example: UDP group has **simple variables + tables of records** (sequence of).

PDU (Protocol Data Unit)

- SNMP works as a **request/reply protocol** using **PDUs** (Protocol Data Units).
- **Types of PDUs:**
 1. **GET:** Manager asks agent for a variable's value.
 2. **GET-NEXT:** Manager asks agent for the next variable in a table.
 3. **SET:** Manager sets a variable's value on the agent.
 4. **RESPONSE:** Agent replies to GET/GET-NEXT with the value.
 5. **TRAP:** Agent sends info to manager about events (e.g., reboot).
- **How it works:**
 1. Manager chooses a variable → sends **PDU request** to agent.
 2. Agent finds the variable → sends **encoded response PDU** back.

PDU type	Request ID	Variable	Value	...
----------	------------	----------	-------	-----

PART A

1. Differentiate application programs and application protocols.

- **API:** Shows valid messages a program can accept; no sequence rules.
- **Protocol:** Defines valid **sequence** of messages between programs.

2. What is WWW and SMTP?

- **WWW:** Internet application to view web pages, store/share data.
- **SMTP:** Protocol for sending emails over TCP/IP.

3. What is a URL, web browser, and rlogin?

- **URL:** Identifier for a web page.
- **Web browser:** Software to display HTML pages.
- **rlogin:** Remote login to access another system.

4. Four main properties of HTTP

- Global URI, request-response exchange, stateless, resource metadata.

5. Four groups of HTTP headers & two HTTP methods

- **Headers:** General, Entity, Request, Response.
- **Methods:** GET, POST.

6. Advantages of persistent TCP connections in HTTP

- Allows **pipelining** → multiple requests without waiting.
- Reduces network congestion.
- TCP can manage congestion better.

7. Uses of HTTP

- Protocol for **distributed hypermedia systems**; foundation of the Web.

8. Usage of conditional GET in HTTP

- Refresh cached data **without resending full data**, saves bandwidth.

9. Why is HTTP stateless?

- Each request is independent → scalable & simpler.

10. Transmission modes of FTP

- Stream mode → continuous data.
- Block mode → data in blocks with headers.
- Compressed mode → files compressed before sending.

11. TCP connections needed in FTP

- Two connections:
 - **Control connection** → simple rules.
 - **Data connection** → handles file transfers.

12. What is TFTP

- Simple file transfer for config/bootstrap; reads/writes files; fits in ROM.

13. Define SMTP

- Protocol for sending emails between users/computers over TCP/IP.

14. Services of a User Agent (UA)

- Compose, read, reply, forward messages; manage mailboxes.

15. Difference between SMTP and MIME

- SMTP → transfers email.
- MIME → allows **non-ASCII files** in email.

16. Why is an application like POP needed?

- To **retrieve emails** from server; SMTP only sends emails.

17. Use of MIME extension

- Converts **non-ASCII/binary files** to ASCII for email.

18. Compare HTTP and FTP

- HTTP → request-response for web.
- FTP → file transfer protocol.

19. Protocol supporting email & basic functions

- **SMTP**: sends emails.
- **Functions**: Compose, Transfer, Report, Display, Dispose.

20. What is POP3

- Client-server protocol to **receive & store emails** on server.

21. Function of POP3

- Downloads emails from server for local access.

22. What is IMAP

- Client-server protocol for accessing emails **on server**; acts like remote file server.

23. Applications of Telnet

- Remote login, network device configuration, online participation.

24. What is Telnet

- TCP/IP protocol for virtual terminal; allows remote access.

25. What is DNS

- Internet phonebook → converts domain names to IP addresses.

26. Information in DNS resource record

- Data about hosts: SOA, NS, A, PTR, CNAME, MX, SRV.

27. Why do we need DNS? Role of DNS resolver

- Maps name ↔ IP.
- Resolver requests DNS server on behalf of clients.

28. Two types of DNS messages

- Query & Response.

29. Define zone

- Part of domain tree a server is **responsible for**.

30. Why are name services called middleware?

- They sit **between apps and network**, managing names.

31. Three main divisions of domain name space

- Generic domains (e.g., .com)
- Country domains (e.g., .in)
- Inverse domain (IP → name)

32. DNS cache issues when IP changes

- Old entries can give wrong pages; **DNS updates** needed.

33. Define Name Resolution

- Process of finding IP from a name (or vice versa).

34. Define SNMP

- Protocol to **manage network devices** like routers, switches, servers.

35. Purpose of SNMP

- Read/write **state info** on network nodes for management.

36. What is MIB

- Collection of **objects managed by an agent**; monitored by management station.
-